Fusion
Technical Project Documentation for a Chat Application

Andy Rugero, Simon Steinschaden, Tristan Hein

February 2026

Contents

1 Project Overview **2**

2 System Architecture **2**
2.1 Architectural Overview 2
2.2 Layered Separation Principle 2

3 Client Architecture (WPF) **3**
3.1 MVVM Pattern 3
3.2 Property Change Notification 3
3.3 Command-Based Interaction 3
3.4 Authentication Flow (Client) 4

4 Server Architecture **4**
4.1 JWT Claim Extraction 4
4.2 Message Endpoint 5

5 Database Design **5**
5.1 Message Entity 5
5.2 Fluent Configuration 6
5.3 Database Persistence 6

6 Security Design **6**
6.1 Conversation Membership Validation 6

7 Migration from JSON to Database **7**

8 Conclusion **7**

1 Project Overview

Fusion is a client-server based chat application developed as a semester project. It consists of:

- A WPF desktop client (MVVM architecture)
- A REST-based ASP.NET Core backend
- A PostgreSQL database

The system supports:

- User registration and login
- JWT-based authentication
- Direct and group conversations
- Persistent message storage

2 System Architecture

Fusion follows a layered client-server architecture with strict separation of concerns.

2.1 Architectural Overview

- Presentation layer (WPF UI)
- Application layer (ViewModels and Services)
- API layer (ASP.NET Controllers)
- Persistence layer (EF Core + PostgreSQL)

All communication between client and server is performed via REST endpoints using JSON-based DTOs. The client never communicates directly with the database.

2.2 Layered Separation Principle

Each layer is isolated:

- UI logic (Views)
- State and behavior (ViewModels)
- HTTP communication (API Services)
- Business logic (Controllers)
- Data persistence (Store implementations)

This architecture allows replacing implementations without affecting higher layers.

3 Client Architecture (WPF)

3.1 MVVM Pattern

The client implements the Model-View-ViewModel pattern.

- Models represent UI-bound data
- ViewModels manage state and commands
- Views are implemented in XAML

3.2 Property Change Notification

```
public class ContactModel : ObservableObject
{
    private string _username;

    public string Username
    {
        get => _username;
        set
        {
            _username = value;
            OnPropertyChanged();
        }
    }
}
```

The base class implements `INotifyPropertyChanged` to ensure UI updates.

3.3 Command-Based Interaction

```
public class AsyncRelayCommand : ICommand
{
    private readonly Func<Task> _execute;
    private readonly Func<bool>? _canExecute;
    private bool _isExecuting;

    public async void Execute(object? parameter)
    {
        if (!CanExecute(parameter))
            return;

        try
        {
            _isExecuting = true;
            RaiseCanExecuteChanged();
            await _execute();
        }
        finally
```

```
        {
            _isExecuting = false;
            RaiseCanExecuteChanged();
        }
    }
}
```

This prevents concurrent command execution during asynchronous operations.

3.4 Authentication Flow (Client)

```
public async Task<LoginResult> LoginAsync(string email, string password)
{
    var req = new LoginRequest
    {
        Email = email.Trim(),
        Password = password
    };

    var dto = await PostJsonAsync<LoginRequest, AuthResponse>("auth/login", req
        );

    if (!dto.Success || string.IsNullOrWhiteSpace(dto.AccessToken))
        return LoginResult.Fail("Invalid credentials.");

    Session.AccessToken = dto.AccessToken;

    _http.DefaultRequestHeaders.Authorization =
        new AuthenticationHeaderValue("Bearer", Session.AccessToken);

    return LoginResult.Ok();
}
```

The JWT token is stored in memory and attached to all subsequent HTTP requests.

4 Server Architecture

4.1 JWT Claim Extraction

```
private static Guid GetUserIdFromClaims(ClaimsPrincipal user)
{
    var raw =
        user.FindFirstValue(ClaimTypes.NameIdentifier) ??
        user.FindFirstValue("sub");

    return Guid.TryParse(raw, out var id) ? id : Guid.Empty;
}
```

Identity is derived strictly from validated JWT claims.

4.2 Message Endpoint

```
[HttpPost("{conversationId:guid}/messages")]
public async Task<ActionResult<MessageDto>> SendMessage(
    Guid conversationId,
    [FromBody] SendMessageRequest req)
{
    var me = GetUserIdFromClaims(User);
    if (me == Guid.Empty) return Unauthorized();

    var content = (req.Content ?? "").Trim();
    if (string.IsNullOrWhiteSpace(content))
        return BadRequest("Empty message.");

    var msg = await _chatStore.AddMessageAsync(conversationId, me, content);

    return Ok(new MessageDto
    {
        MessageId = msg.MessageId,
        ConversationId = msg.ConversationId,
        SenderUserId = msg.SenderUserId,
        Content = msg.Content,
        SentAtUtc = msg.SentAtUtc
    });
}
```

Validation steps:

- Authentication verification
- Content validation
- Membership validation

5 Database Design

5.1 Message Entity

```
public class Message
{
    public Guid Id { get; set; }

    public Guid ConversationId { get; set; }
    public Conversation? Conversation { get; set; }

    public Guid SenderUserId { get; set; }
    public User? SenderUser { get; set; }

    public DateTime SentAtUtc { get; set; }
    public string Content { get; set; } = string.Empty;
}
```

5.2 Fluent Configuration

```
modelBuilder.Entity<Message>()
    .HasOne(m => m.Conversation)
    .WithMany(c => c.Messages)
    .HasForeignKey(m => m.ConversationId)
    .OnDelete(DeleteBehavior.Cascade);

modelBuilder.Entity<Message>()
    .HasOne(m => m.SenderUser)
    .WithMany()
    .HasForeignKey(m => m.SenderUserId)
    .OnDelete(DeleteBehavior.Restrict);
```

5.3 Database Persistence

```
public async Task<MessageRecord> AddMessageAsync(
    Guid conversationId,
    Guid senderUserId,
    string content)
{
    var message = new Message
    {
        Id = Guid.NewGuid(),
        ConversationId = conversationId,
        SenderUserId = senderUserId,
        Content = content,
        SentAtUtc = DateTime.UtcNow
    };

    _db.Messages.Add(message);
    await _db.SaveChangesAsync();

    return ToRecord(message);
}
```

EF Core ensures transactional consistency and referential integrity.

6 Security Design

6.1 Conversation Membership Validation

```
if (!convo.ParticipantUserIds.Contains(senderUserId))
    throw new InvalidOperationException(
        "Sender is not a participant of this conversation.");
```

This prevents unauthorized message injection.

7 Migration from JSON to Database

The project initially used JSON-based file storage. Limitations included lack of foreign key constraints, full file rewrites and potential race conditions.

Replacing:

- IUserStore → DatabaseUserStore
- IChatStore → DatabaseChatStore

required no changes to controller logic due to interface abstraction.

8 Conclusion

Fusion demonstrates a structured and extensible client-server architecture built with modern software engineering principles. The project combines MVVM on the client side, layered API design on the server side, and relational persistence using PostgreSQL.

A strong emphasis was placed on separation of concerns. The introduction of abstraction layers such as `IUserStore` and `IChatStore` allowed the persistence implementation to evolve from JSON-based storage to a fully relational PostgreSQL database without requiring changes to higher-level business logic. This architectural decision significantly improved scalability, maintainability and data integrity.

Security considerations were integrated throughout the system. Authentication is handled via JWT tokens, while authorization checks ensure that users can only access conversations in which they are members. Database-level foreign key constraints complement application-level validation, resulting in a layered security model.

While the current implementation focuses on reliability and clean architecture, the system is designed to support future extensions. Potential improvements include:

- Integration of real-time communication via WebSockets or SignalR
- Implementation of PostgreSQL Row-Level Security (RLS) for database-level authorization
- Introduction of message editing and soft deletion policies
- Expansion of role-based permissions within group conversations

These enhancements can be implemented without fundamental architectural changes, demonstrating that Fusion provides a robust and extensible foundation for a secure messaging platform.

Overall, the project reflects deliberate architectural decisions, security-aware design, and a clear focus on maintainability and scalability.